AI Mental Health Companion

Imagine you're building a digital friend — one that listens when you're having a rough day, understands how you're feeling, and gently suggests something helpful, like a breathing exercise or a journaling prompt.

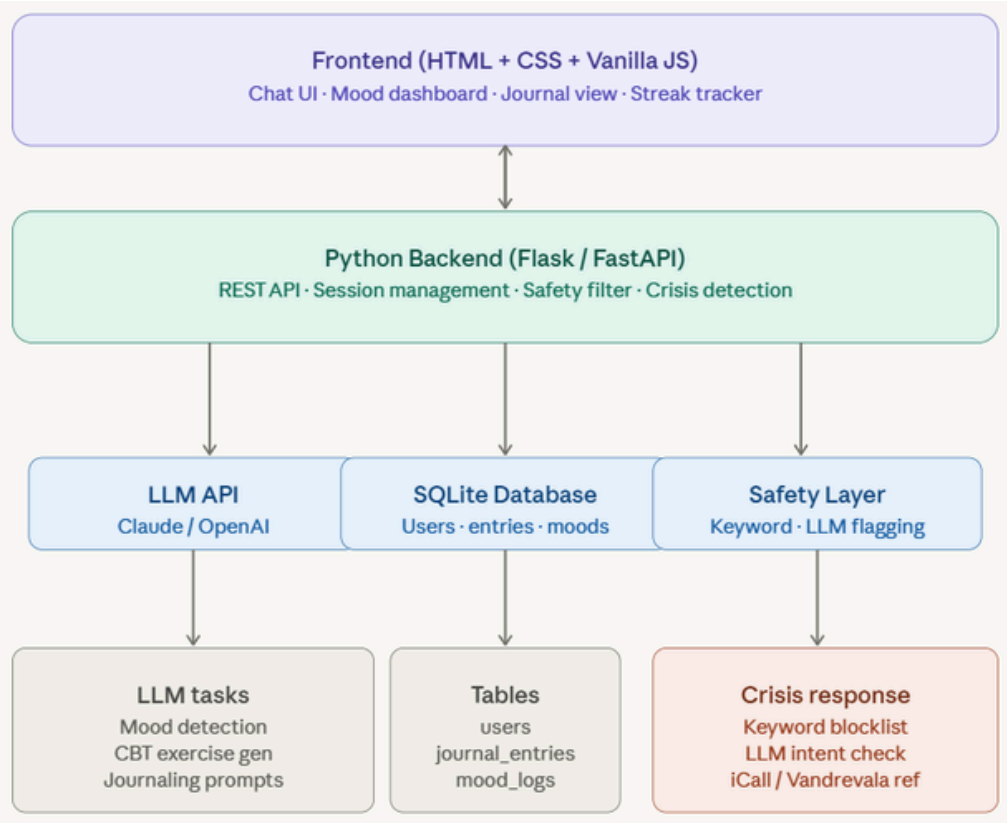
Here's how it works in plain words:
The user opens the app and types how they're feeling — maybe "I've been really stressed about exams lately." Your app reads that, quietly figures out the emotion behind it (stressed, anxious, sad), and then responds like a caring friend would. Not robotic. Not clinical. Just warm and human.

Behind the scenes, an LLM (like Claude or ChatGPT) is doing the heavy lifting — reading the message, understanding the mood, and crafting a helpful reply along with a small exercise, like "Try this 4-7-8 breathing

technique."
Every conversation gets saved, so over time the app can show the user a mood chart — like "hey, you've been feeling better this week!" That little streak of progress can genuinely motivate people.
The one thing you absolutely can't skip is the safety layer. If someone types something that sounds like they're in crisis, the app must immediately stop everything and show a helpline number. No AI response. No exercise. Just "here's a number, please call." This is the most important part of the whole project.

What to build first (MVP order)

1. **Basic chat UI + Python backend with LLM API connected**
2. **Mood detection from user messages (prompt engineering)**
3. **Journal entry storage with SQLite**
4. **CBT exercise suggestions based on mood**
5. **Streak/history dashboard**
6. **Crisis keyword detection with helpline redirect**



Smart Resume & Job Match Analyzer

Imagine you're applying for a job. You spend hours on your resume, hit submit — and hear nothing back. Why? Maybe your resume had the right experience but the wrong words. Maybe a computer (called an ATS) filtered it out before any human ever saw it.

Here's what it does in plain words:
You upload your resume and paste the job description. The app reads both — like a smart friend who knows hiring — and tells you exactly what's matching, what's missing, and what to fix. It even rewrites your weak bullet points for you. Then you download the improved resume and apply with confidence.

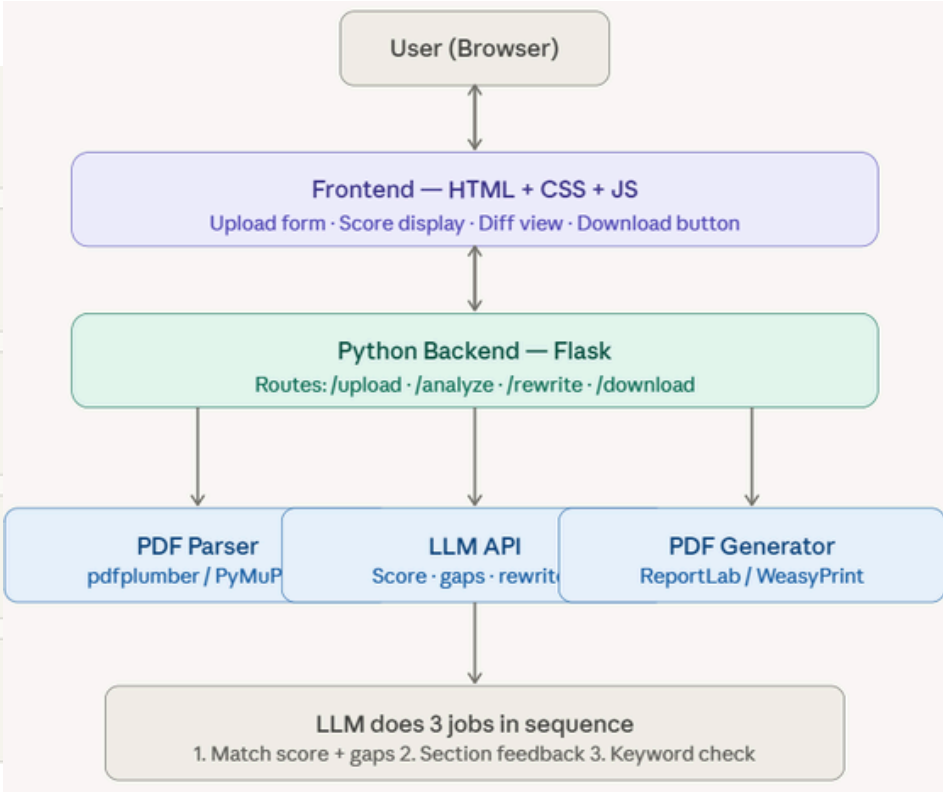
Your three tools each have one job:
Python is the engine — it reads your PDF, talks to the AI, and builds the new resume file. HTML and CSS are the face — the upload form, the score display, the before/after view. The LLM is the brain — it compares your resume to the job, finds the gaps, and writes better versions of your bullet points.

The one thing that makes or breaks this app is the prompt you send to the AI. You need to ask it very specifically — "compare these two texts, give me a score, list what's missing, and rewrite the weak lines" — and tell it to reply in a structured format so your code can read it cleanly. Get that prompt right, and the whole app comes together.

The biggest headache you'll face is that PDFs are messy. A two-column resume or a fancy template can confuse the PDF reader and give you garbage text. Always let users paste their resume as plain text too — that's your safety net.

What the user feels when they use it:
They come in frustrated and unsure. They leave with a score, clear feedback, and a better resume in hand. That feeling — going from "why isn't this working?" to "okay, now I know what to fix" — is exactly what makes this project worth building.

1. **File upload + text extraction**
   Accept PDF resume, pull out the raw text
2. **Match score calculation**
   Send resume + JD to LLM, get a score back
3. **Section-by-section feedback**
   Weak lines get flagged and rewritten
4. **ATS keyword optimization**
   Flag missing keywords that ATS systems look for
5. **Download improved resume**
   Export the rewritten resume as a PDF



AI Legal Document Simplifier

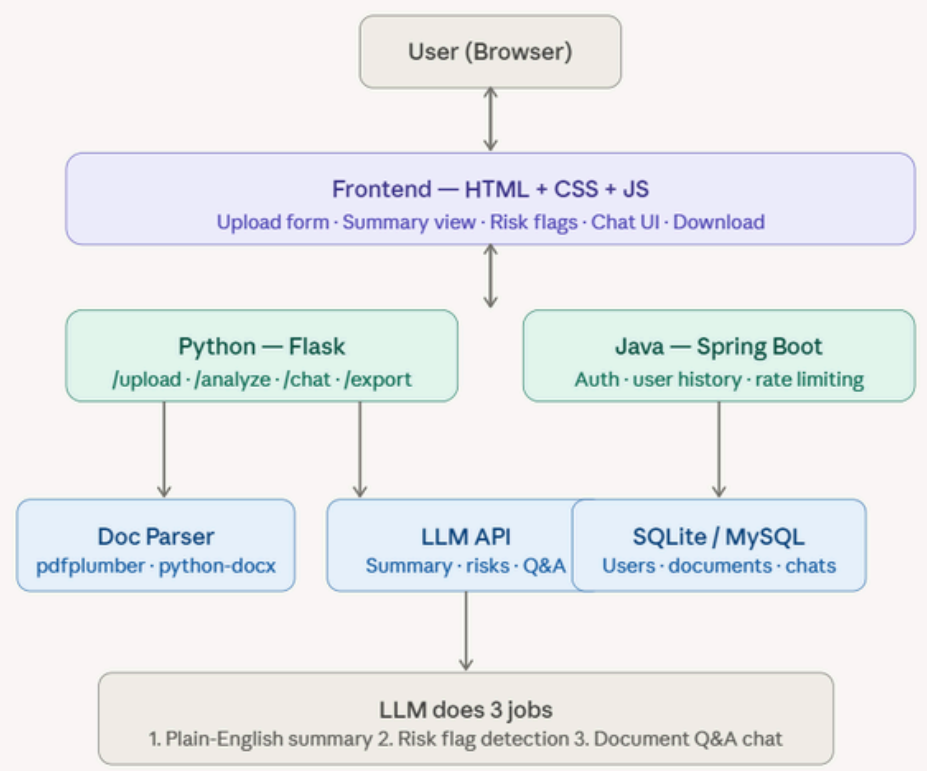
Think about the last time someone handed you a long document and said "just sign here."
A rental agreement. An internship offer letter. App terms and conditions. You scrolled through pages of dense, confusing text — words like "indemnification," "arbitration clause," "unilateral termination rights" — and thought, I'll just trust it and sign.
Most people do. Not because they're careless. Because understanding legal language feels like learning a foreign language nobody taught you.

What it actually does:
You upload the document. The app reads it — every clause, every buried condition — and explains it back to you in plain, friendly language. Like a smart friend who happened to go to law school, sitting next to you and saying "okay, this part means they can raise your rent with just 7 days notice — that's worth knowing."
It also highlights the risky parts in red. Not every clause is dangerous, but some are quietly unfair — hidden auto-renewals, one-sided exit conditions, things that only matter when something goes wrong. The app finds those and flags them clearly.
And then you can just... ask it questions. "Can they enter my room without permission?" "What happens if I leave before the lease ends?" It answers directly, using only what's written in the document — not guessing, not making things up.

Your three tools each have a clear job:
Python reads the document and runs all the AI logic. HTML and CSS build the interface — the upload screen, the summary, the risk highlights, the chat box. Java handles the behind-the-scenes stuff — user accounts, login, saving your document history so you can come back to it later.

What the user feels when they use it:
They come in anxious and confused. They leave actually understanding what they're about to sign. That shift — from powerless to informed — is exactly why this project is worth building.

1. **Document upload + text extraction**
   Accept PDF or DOCX, pull out the raw text
2. **Plain English summary**
   LLM translates the whole document simply
3. **Risk flag detection**
   Highlight clauses that look unfair or dangerous
4. **Q&A chat about the document**
   User asks questions, AI answers from the doc
5. **Export summary as PDF**
   Download a clean simplified version



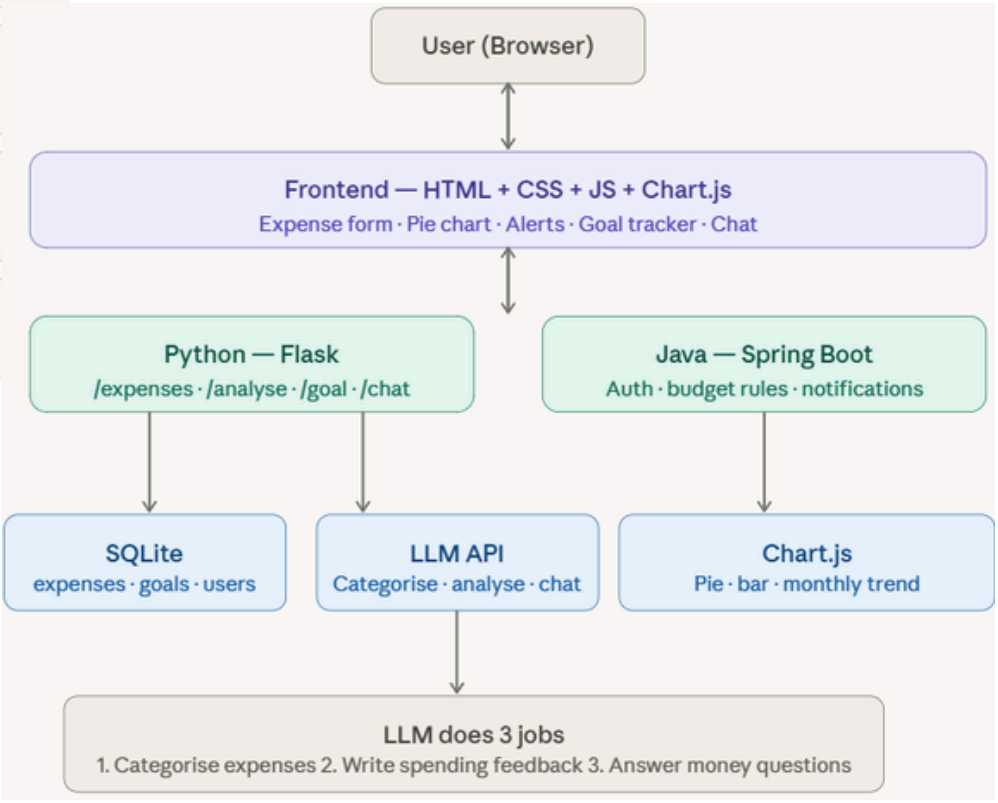
AI Personal Finance Coach

The real problem this solves:
Most people have no idea where their money actually goes. They earn, they spend, the month ends — and somehow there's less than expected. Not because they're reckless, but because nobody ever taught them to track it properly, and spreadsheets feel like homework. This app does the tracking, finds the patterns, and talks to them about it like a real coach would.

Here's what it does in plain words:
You enter what you earned and what you spent this month. The app reads it, sorts your expenses into categories automatically, spots where you're overspending, suggests a savings plan, and lets you ask it questions like "can I afford a new phone?" or "how long to save ₹50,000?" You get honest, personalised answers — not generic advice from a blog post.

In plain English — what the user feels
They open the app and log this month's expenses — takes about two minutes. Instantly they see a pie chart of where their money went, written feedback from their AI coach, and a clear answer to "am I on track to save ₹20,000 by December?" They ask a follow-up question, get a specific answer using their actual numbers, and leave with a plan they actually understand.
That shift — from "I have no idea where my money goes" to "okay, I need to cut ₹1,500 from food and I'll hit my goal" — is exactly what makes this project worth building.
Want me to write the actual code for any specific part — the Flask routes, the categorisation function, the Chart.js dashboard, or the savings goal tracker?

1. **Expense entry + storage**
   Let users log what they spent and earned
2. **Auto-categorisation with LLM**
   AI sorts expenses into food, rent, transport etc.
3. **Spending analysis + overspend alerts**
   Find where money is leaking out
4. **Savings goal + AI milestones**
   Set a goal, get a realistic plan to reach it
5. **Finance Q&A chatbot**
   Ask anything about your own money



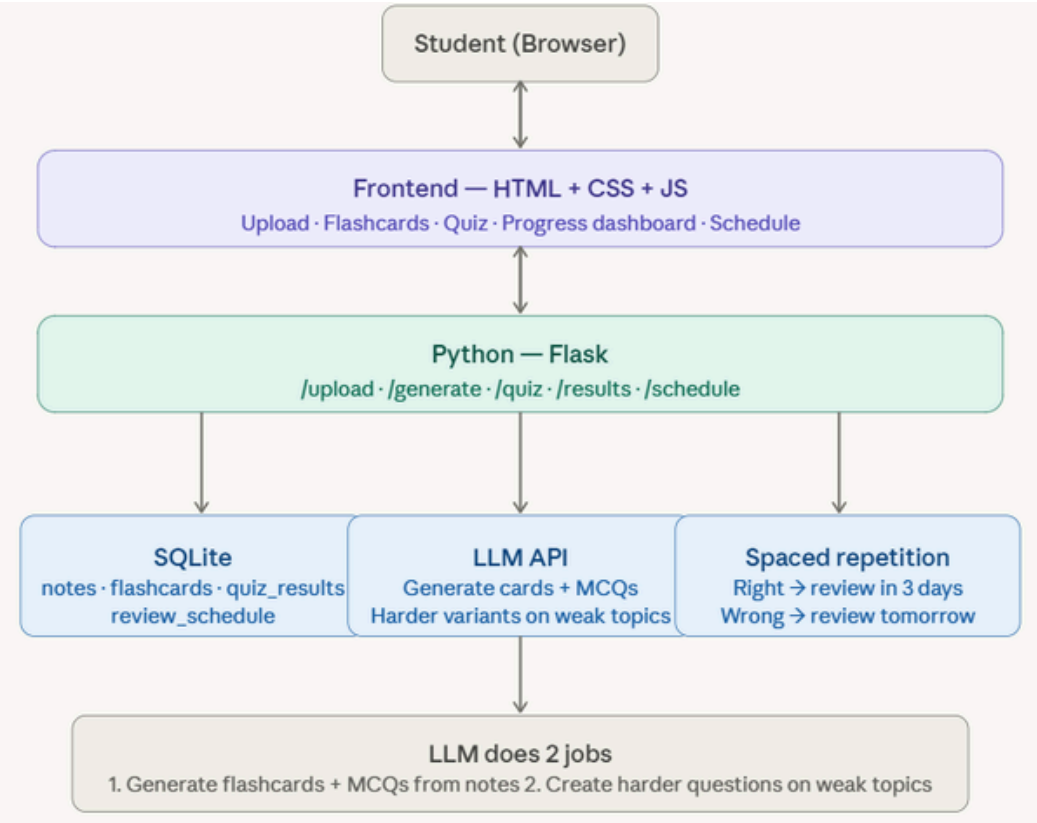
Personalized Study Buddy

The real problem this solves:
Every student has been there. Exam in two days. Notes open. Highlighter in hand. Re-reading the same paragraph for the third time, hoping something sticks. The problem isn't laziness — it's that passive reading is one of the worst ways to actually learn. Testing yourself, spacing out your practice, being forced to recall — that's what actually works. This app does all of that automatically, from your own notes.

Here's what it does in plain words:
You paste your notes or upload a chapter. The app reads it, generates flashcards and quiz questions from it, figures out which topics you're weakest on based on your answers, and builds you a study schedule that focuses your time where it matters most. Like having a personal tutor who read your exact notes and knows exactly where you're struggling.

1. **Notes upload + text extraction**
   Accept typed notes or a PDF, pull out clean text
2. **Flashcard + quiz generation**
   LLM reads notes and creates questions automatically
3. **Quiz mode + performance tracking**
   User answers, app records right and wrong
4. **Adaptive difficulty + weak topic focus**
   Show harder questions on topics the user keeps getting wrong
5. **Spaced repetition schedule**
   Tell users exactly what to review and when

What the user walks away with:
Not just a quiz. A system that knows what they know, knows what they don't, and tells them exactly what to study today. That's the difference between a study tool and a study buddy.



AI Mental Health Companion

A conversational app that detects mood from user text and offers evidence-based CBT exercises, breathing guides, and crisis escalation.

Health Intermediate

- **Tech stack:** Python (Flask/FastAPI), LLM (Claude/GPT), HTML + CSS, SQLite
- **Key features:** Sentiment & mood analysis using LLM · Personalized journaling prompts · Crisis keyword detection with helpline redirect · Streak tracking and mood charts
- **LLM role:** LLM reads journal entries to detect emotional state and suggest tailored CBT techniques.

Smart Resume & Job Match Analyzer

Paste a resume and job description — AI identifies gaps, rewrites weak sections, and gives a match score with actionable feedback.

Productivity Beginner

- **Tech stack:** Python, LLM API, HTML + CSS, PDF parsing (PyMuPDF)
- **Key features:** Resume vs JD gap analysis · Section-by-section rewrite suggestions · ATS keyword optimization · Download improved resume as PDF
- **LLM role:** LLM compares resume content to job description semantics and generates specific rewrites.

AI Legal Document Simplifier

Upload any legal document (rent agreement, terms of service, employment contract) and get a plain-English summary with risk highlights.

Legal & Finance Intermediate

- **Tech stack:** Python, LLM API, Java (Spring Boot backend), HTML + CSS
- **Key features:** Clause-by-clause plain English translation · Risk flag detection (unfair clauses) · Q&A chat about the document · Export summary as PDF
- **LLM role:** LLM identifies risky clauses, translates legalese, and answers follow-up questions in context.

AI Personal Finance Coach

Users input their monthly expenses and income — AI finds patterns, warns about overspending, and suggests a savings plan with investment basics.

Legal & Finance Intermediate

- **Tech stack:** Python, LLM API, Java (backend logic), HTML + CSS + Chart.js
- **Key features:** Expense categorization with NLP · Overspend alerts and budget suggestions · Savings goal setting with AI milestones · Q&A chatbot for finance questions
- **LLM role:** LLM interprets spending patterns in natural language and generates personalized financial advice.

Personalized Study Buddy

A student uploads notes or a textbook chapter — AI generates flashcards, quizzes, and a study schedule tailored to their weak areas.

Education Intermediate

- **Tech stack:** Python, LLM API, HTML + CSS, Local Storage / SQLite
- **Key features:** Auto flashcard & MCQ generation from notes · Spaced repetition scheduling · Adaptive difficulty based on quiz performance · Progress dashboard
- **LLM role:** LLM generates diverse question types from raw content and adapts based on wrong answer